```
// State & Domain Question Bank
const QUESTION_BANK = {
  html: [
    {
      id: 1,
      q: "Which HTML tag defines an internal style sheet?",
      opts: ["<css>", "<script>", "<style>", "<sheet>"],
      ans: "<style>",
      exp: "<style> tag encapsulates CSS declarations within an HTML document."
    },
    {
      id: 2,
      q: "Which attribute specifies alternative text for images?",
      opts: ["title", "alt", "src", "desc"],
      ans: "alt",
      exp: "alt provides accessible text descriptions for assistive technology."
    }
  ],
  css: [
    {
      id: 3,
      q: "Which property controls element background fill?",
      opts: ["color", "bgcolor", "background-color", "fill"],
      ans: "background-color",
      exp: "background-color sets the background canvas fill."
    }
  ],
  javascript: [
    {
      id: 4,
      q: "Which method converts a JSON string into a JavaScript object?",
      opts: ["JSON.stringify()", "JSON.parse()", "JSON.toObject()", "JSON.build()"],
      ans: "JSON.parse()",
      exp: "JSON.parse() deserializes a standard JSON string into a JS object."
    }
  ],
  architecture: [
    {
      id: 5,
      q: "Which HTTP status code signifies a successful REST request?",
      opts: ["200 OK", "404 Not Found", "500 Server Error", "301 Moved"],
      ans: "200 OK",
      exp: "200 OK indicates that the client request has succeeded."
    }
  ]
};

const CATEGORIES = [
  { key: 'html', title: 'HTML5 Fundamentals', desc: 'Semantic tags, document structure, forms, and web
markup.' },
  { key: 'css', title: 'CSS3 Styling & Layouts', desc: 'Flexbox, CSS Grid, animations, selectors, and
glassmorphism.' },
  { key: 'javascript', title: 'JavaScript ES6+', desc: 'DOM manipulation, closures, events, promises, and
array methods.' },
  { key: 'architecture', title: 'Web Architecture', desc: 'HTTP protocols, client-server models, Web APIs, and
caching.' }
];

let currentCat = null;
let activeQs = [];
let userAnswers = {};
let timeRemaining = 300;
let timerId = null;

// Fisher-Yates Algorithmic Randomization (O(n) In-Place Shuffling)
function shuffleArray(arr) {
  const clone = [...arr];
  for (let i = clone.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [clone[i], clone[j]] = [clone[j], clone[i]];
  }
  return clone;
```

```
// Category Selection & Quiz Initialization
function selectCategory(key) {
  currentCat = key;
  userAnswers = {};
  timeRemaining = 300;
  activeQs = shuffleArray(QUESTION_BANK[key] || QUESTION_BANK.html).map(item => ({
    ...item,
    opts: shuffleArray(item.opts)
  }));

  document.getElementById('categorySection').classList.add('hidden');
  document.getElementById('resultSection').classList.add('hidden');
  document.getElementById('quizSection').classList.remove('hidden');

  updateProgress();
  renderQuestions();
  startCountdown();
}

// Render Questions to Active DOM
function renderQuestions() {
  const wrapper = document.getElementById('questionswrapper');
  wrapper.innerHTML = '';

  activeQs.forEach((item, qIdx) => {
    const qCard = document.createElement('div');
    qCard.className = 'q-card';
    const safeQuestion = item.q.replace(/</g, "&lt;").replace(/>/g, "&gt;");

    let optionsHTML = '';
    item.opts.forEach((opt) => {
      const safeOpt = opt.replace(/</g, "&lt;").replace(/>/g, "&gt;");
      optionsHTML += `
        <label class="option-label" id="label_${qIdx}_${opt}">
          <input type="radio" name="q_${qIdx}" value="${opt}" onchange="selectOption(${qIdx}, '${opt}')">
          <span>${safeOpt}</span>
        </label>
      `;
    });

    qCard.innerHTML = `<h4>Q${qIdx + 1}: ${safeQuestion}</h4><div class="options-list">${optionsHTML}</div>`;
    wrapper.appendChild(qCard);
  });
}

function selectOption(qIdx, selectedValue) {
  userAnswers[qIdx] = selectedValue;
  const labels = document.querySelectorAll(`input[name="q_${qIdx}"]`);
  labels.forEach(input => {
    input.closest('.option-label').classList.toggle('selected', input.value === selectedValue);
  });
  updateProgress();
}

function updateProgress() {
  const answeredCount = Object.keys(userAnswers).length;
  const total = activeQs.length;
  document.getElementById('answeredCount').textContent = `${answeredCount}/${total}`;
  const pct = total > 0 ? (answeredCount / total) * 100 : 0;
  document.getElementById('progressBar').style.width = `${pct}%`;
}

// Sticky Countdown Timer & Auto-Submit Trigger
function startCountdown() {
  clearInterval(timerId);
  const badge = document.getElementById('timerBadge');
  badge.classList.remove('timer-warning');

  timerId = setInterval(() => {
    timeRemaining--;
    const min = String(Math.floor(timeRemaining / 60)).padStart(2, '0');
    const sec = String(timeRemaining % 60).padStart(2, '0');
```

```
// Modal Safeguard Handlers
function handlePreSubmission() {
  const count = Object.keys(userAnswers).length;
  document.getElementById('modalStatusText').textContent = `You have answered ${count} of ${activeQs.length}
questions.`;
  document.getElementById('confirmModal').classList.remove('hidden');
}

function closeModal() {
  document.getElementById('confirmModal').classList.add('hidden');
}

function confirmSubmit() {
  closeModal();
  finalizeQuiz(false);
}

// Evaluation Engine & Persistent High Score Synchronization
function finalizeQuiz(autoSubmitted = false) {
  clearInterval(timerId);
  let correct = 0;
  const attempted = Object.keys(userAnswers).length;

  activeQs.forEach((q, idx) => {
    if (userAnswers[idx] === q.ans) correct++;
  });

  const percentage = Math.round((correct / activeQs.length) * 100);
  const bestKey = `quiz_best_${currentCat}`;
  const previousBest = parseInt(localStorage.getItem(bestKey) || '0', 10);

  if (percentage > previousBest) {
    localStorage.setItem(bestKey, percentage);
  }

  renderResultsView(correct, activeQs.length, attempted, percentage, autoSubmitted);
}

// Render Result & Pedagogical Review
function renderResultsView(correct, total, attempted, percentage, autoSubmitted) {
  document.getElementById('quizSection').classList.add('hidden');
  const resultSection = document.getElementById('resultSection');
  resultSection.classList.remove('hidden');

  const resultCard = document.getElementById('resultCard');
  resultCard.innerHTML = `
    <h2>${autoSubmitted ? 'Time Expired!' : 'Congratulations!'}</h2>
    <div style="font-size: 2.5rem; font-weight: bold; margin: 12px 0;">${correct}/${total}</div>
    <div class="badge" style="display:inline-block; margin-bottom: 16px;">
      ${percentage >= 50 ? 'PASSED' : 'NEEDS IMPROVEMENT'} • ${percentage}%
    </div>
    <p>Attempted: ${attempted} | Accuracy: ${attempted > 0 ? Math.round((correct / attempted) * 100) : 0}%</p>
    <button class="btn-primary" onclick="initApp();
document.getElementById('categorySection').classList.remove('hidden');
document.getElementById('resultSection').classList.add('hidden');">
      Return to Categories
    </button>
  `;

  const reviewContainer = document.getElementById('reviewContainer');
  reviewContainer.innerHTML = '<h3>Answer Review</h3>';

  activeQs.forEach((q, idx) => {
    const isCorrect = userAnswers[idx] === q.ans;
    const item = document.createElement('div');
    item.className = 'review-item';
    item.innerHTML = `
      <p><strong>Q${idx + 1}: ${q.q.replace(/</g, "&lt;")}</strong></p>
      <p style="color: ${isCorrect ? '#10b981' : '#ef4444'};">
        Your Answer: ${userAnswers[idx] ? userAnswers[idx].replace(/</g, "&lt;") : '<em>Unattempted</em>'}
      </p>
      <p style="color: #60a5fa;">Correct Answer: ${q.ans.replace(/</g, "&lt;")}</p>
```